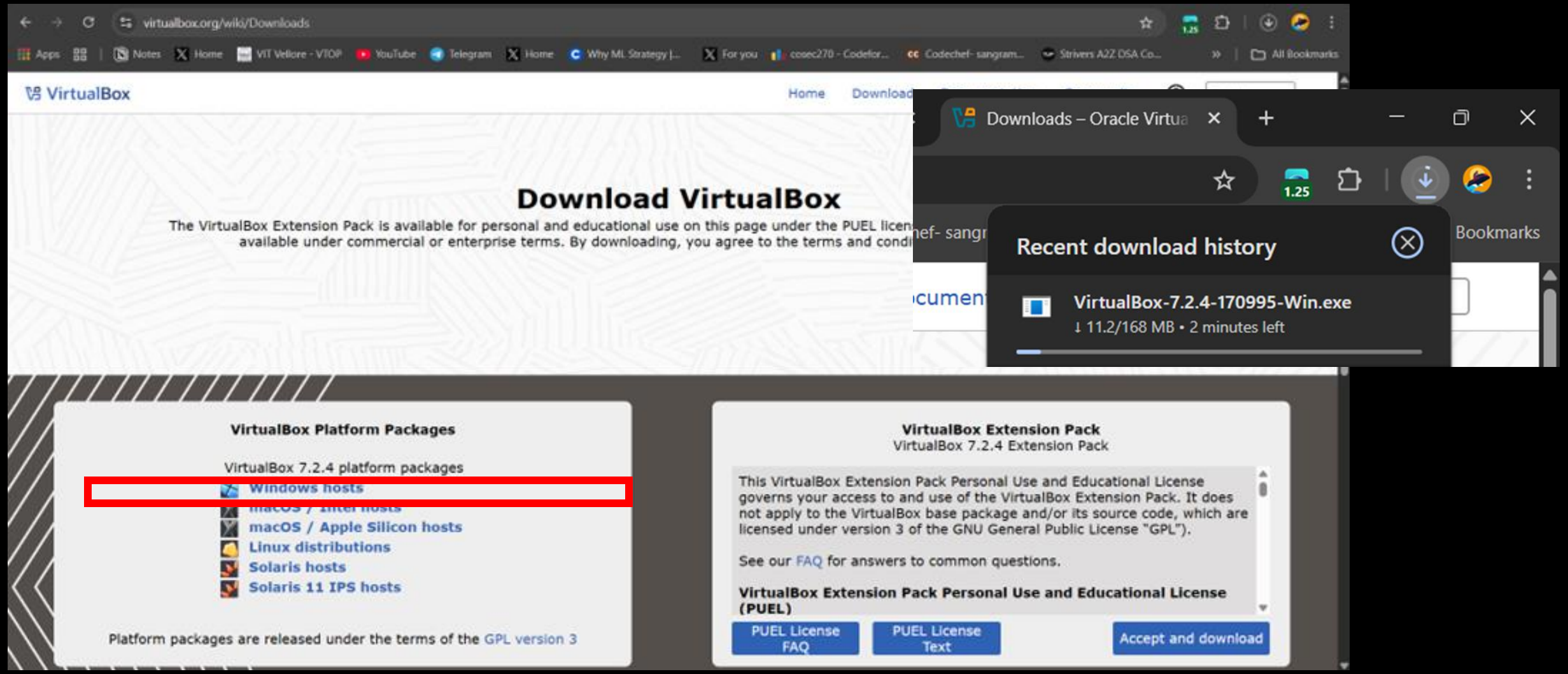
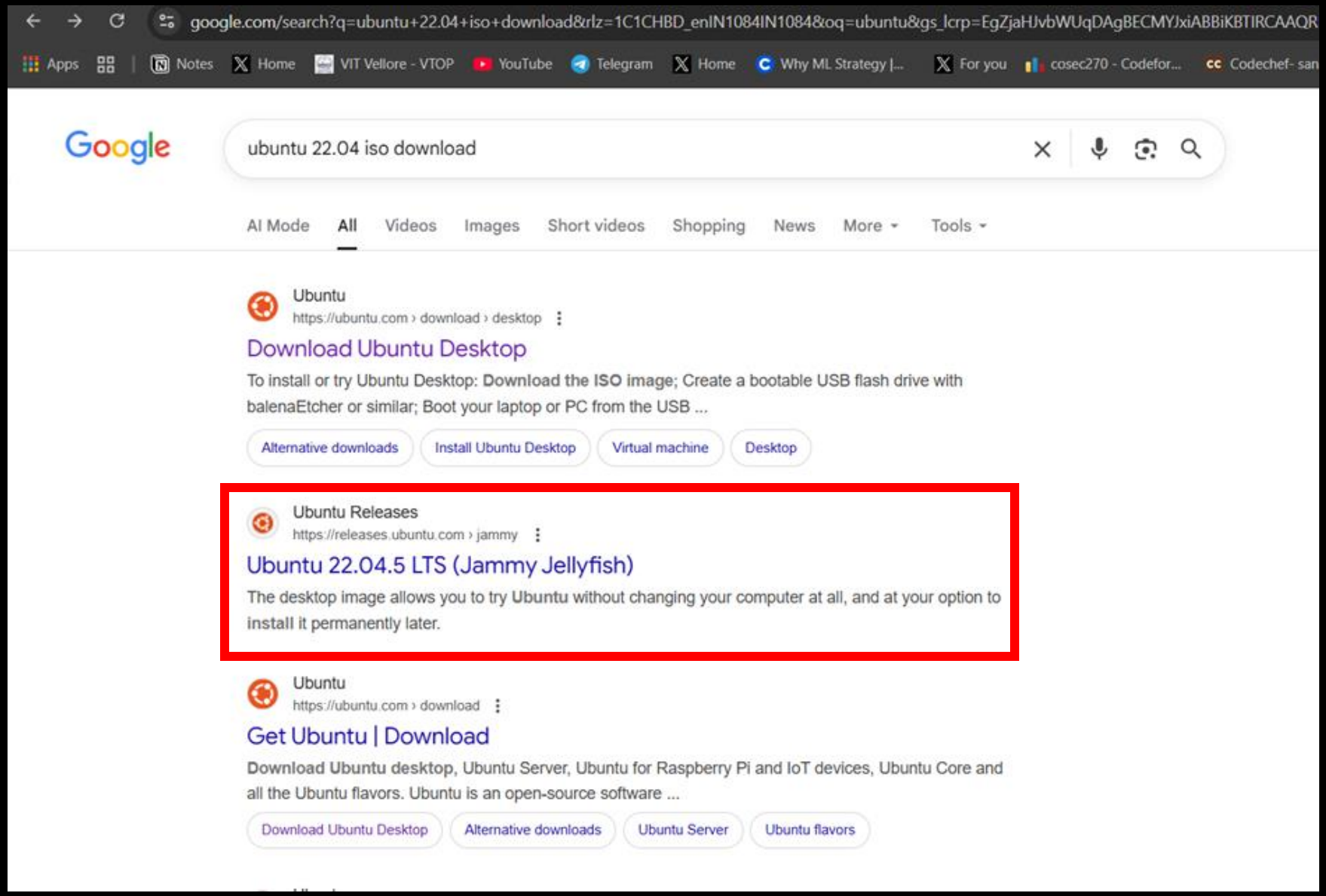
Part A

Install Virtual Machine and Ubuntu 22.04

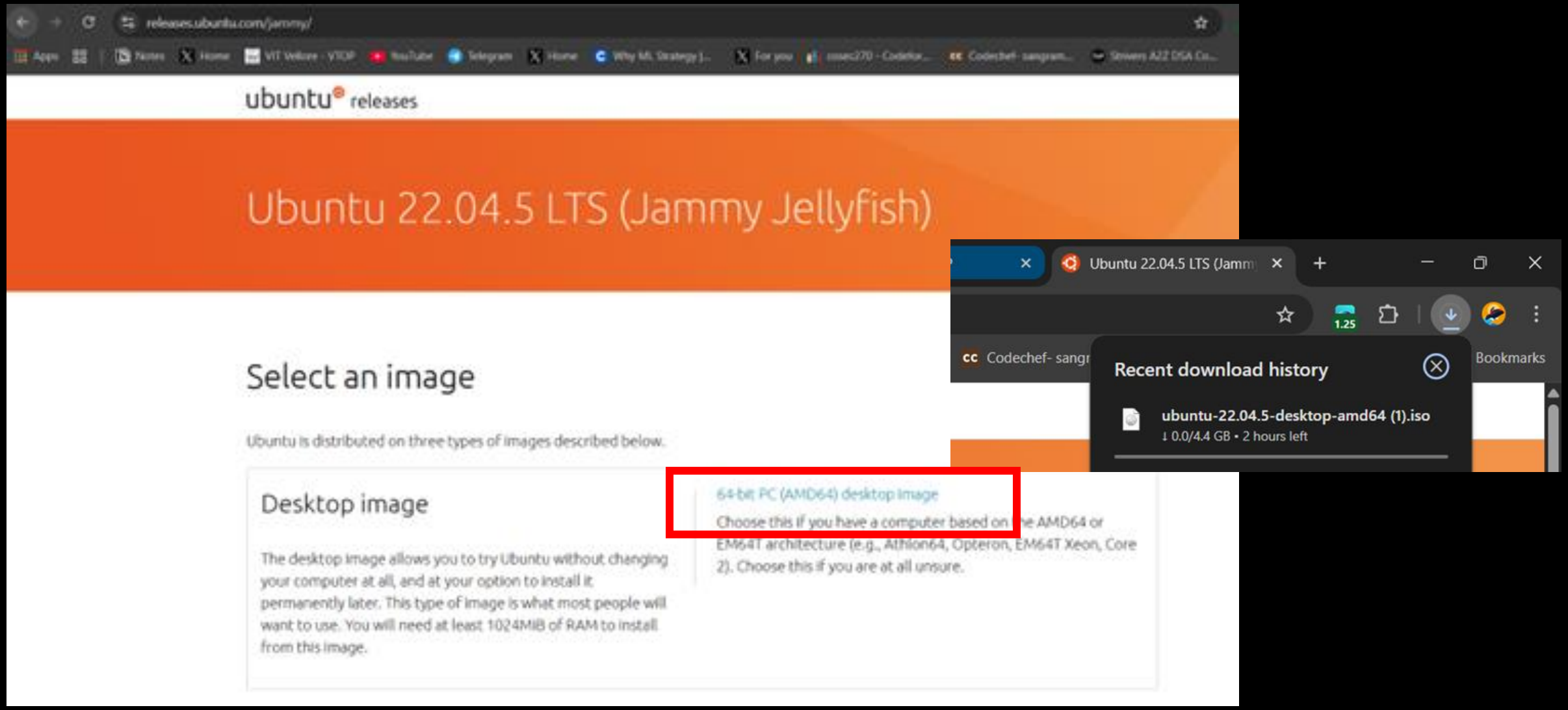
Download Oracle VM VirtualBox from https://www.virtualbox.org/wiki/Downloads



Search 'ubuntu 22.04 iso download' click on the highlighted link



Link to download Ubuntu 22.04.5: https://releases.ubuntu.com/jammy/



Once everything is downloaded make sure to set the settings to same as these

General

Name: Ubuntu_22.04
Operating System: Ubuntu (64-bit)

System

Base Memory: 8192 MB
Processors: 4
Boot Order: Floppy, Optical, Hard Disk
Acceleration: Nested Paging, KVM Paravirtualization

Display

Video Memory: 16 MB
Graphics Controller: VMSVGA
Remote Desktop Server: Disabled
Recording: Disabled

Storage

Controller: IDE
IDE Secondary Device 0: [Optical Drive] Empty
Controller: SATA
SATA Port 0: Ubuntu_22.04.vdi (Normal, 25.00 GB)

Audio

Host Driver: Default
Controller: ICH AC97

Network

Adapter 1: Intel PRO/1000 MT Desktop (NAT)

USB

USB Controller: OHCI, EHCI
Device Filters: 0 (0 active)

Shared folders

None

Description

None

Your system is now ready to proceed with OpenRAM installation. Open the terminal in Ubuntu and follow the next steps.

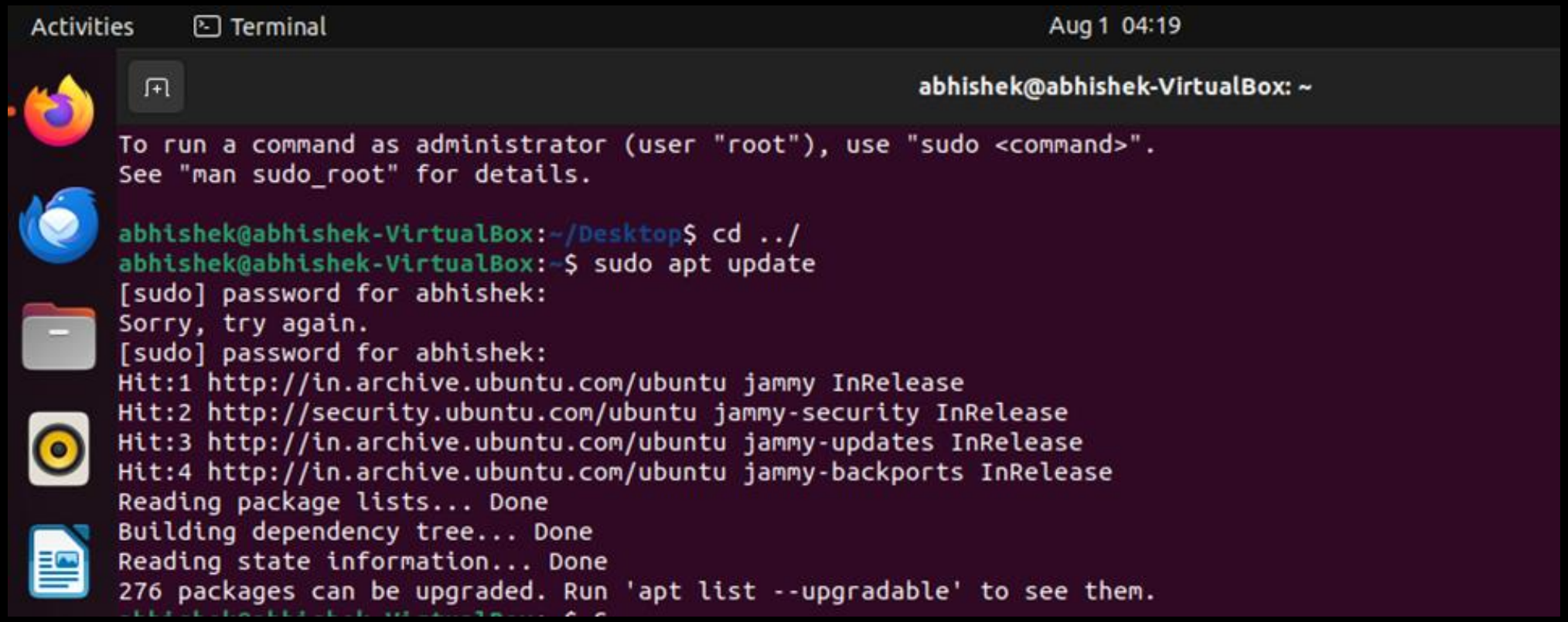
PART B

Steps to install OpenRAM and to design a memory

(here we've designed a 1kb memory file)

(Short Cuts: ctrl+O+Enter → Save, ctrl+X → exit, ctrl+shift+C/V → copy/paste)
(Make sure you are using Ubuntu 22.04 or 20.04 only. Other versions are NOT compatible with OpenRAM)

1.Install required Ubuntu dependencies by typing the command: **sudo apt update**



3. Install the necessary libraries needed alongside OpenRAM using the command:

pip3 install numpy matplotlib

```
abhishek@abhishek-VirtualBox:~$ pip3 install numpy matplotlib
Defaulting to user installation because normal site-packages is not writeable
Collecting numpy
  Downloading numpy-2.2.6-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (16.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.8/16.8 MB 3.7 MB/s eta 0:00:00
Collecting matplotlib
  Downloading matplotlib-3.10.5-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (8.7 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.7/8.7 MB 4.3 MB/s eta 0:00:00
Collecting contourpy>=1.0.1
  Downloading contourpy-1.3.2-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (325 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 325.0/325.0 KB 3.2 MB/s eta 0:00:00
Collecting fonttools>=4.22.0
  Downloading fonttools-4.59.0-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (4.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.8/4.8 MB 5.2 MB/s eta 0:00:00
Collecting cycler>=0.10
  Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/lib/python3/dist-packages (from matplotlib) (2.4.7)
Collecting packaging>=20.0
  Downloading packaging-25.0-py3-none-any.whl (66 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 66.5/66.5 KB 2.5 MB/s eta 0:00:00
Collecting kiwisolver>=1.3.1
  Downloading kiwisolver-1.4.8-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64.whl (1.6 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 4.4 MB/s eta 0:00:00
Requirement already satisfied: pillow>=8 in /usr/lib/python3/dist-packages (from matplotlib) (9.0.1)
Requirement already satisfied: python-dateutil>=2.7 in /usr/lib/python3/dist-packages (from matplotlib) (2.8.1)
Installing collected packages: packaging, numpy, kiwisolver, fonttools, cycler, contourpy, matplotlib
  WARNING: The scripts f2py and numpy-config are installed in '/home/abhishek/.local/bin' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
  WARNING: The scripts fonttools, pyftmerge, pyftsubset and ttx are installed in '/home/abhishek/.local/bin' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed contourpy-1.3.2 cycler-0.12.1 fonttools-4.59.0 kiwisolver-1.4.8 matplotlib-3.10.5 numpy-2.2.6 packaging-25.0
abhishek@abhishek-VirtualBox:~$
```

4. Clone and install OpenRAM by typing: **git clone https://github.com/VLSIDA/OpenRAM.git**

```
Successfully installed contourpy-1.3.2 cycler-0.12.1 fonttools-4.59.0 kiwisolver-1.4.8 matplotlib-3.10.5 numpy-2.2.6 packaging-25.0
abhishek@abhishek-VirtualBox:~$ git clone https://github.com/VLSIDA/OpenRAM.git
Cloning into 'OpenRAM'...
remote: Enumerating objects: 40794, done.
remote: Counting objects: 100% (9699/9699), done.
remote: Compressing objects: 100% (419/419), done.
remote: Total 40794 (delta 9482), reused 9284 (delta 9279), pack-reused 31095 (from 3)
Receiving objects: 100% (40794/40794), 74.08 MiB | 5.12 MiB/s, done.
Resolving deltas: 100% (32900/32900), done.
abhishek@abhishek-VirtualBox:~$ S
```

OpenRAM is successfully cloned and ready to use.

5. **cd openRAM :***guides into the OpenRAM repository*

ls *:lists all the directories and files available*

```
abhishek@abhishek-VirtualBox:~$ cd OpenRAM
abhishek@abhishek-VirtualBox:~/OpenRAM$ ls
CNAME        CONTRIBUTING.md  __init__.py      Makefile                    PORTING.md       rom_compiler.py  sram_compiler.py
common.py    docker           install_conda.sh MANIFEST.in                 pyproject.toml   setpaths.sh      sram_func.py
compiler     docs             LICENSE          OpenRAM_ICCAD_2016_paper.pdf README.md       setup.py         technology
_config.yml  images           macros           openram.mk                  requirements.txt sram_char.py     VERSION
abhishek@abhishek-VirtualBox:~/OpenRAM$
```

6. Create a python file by typing **nano sram_1kb.py** *in the terminal. This creates and takes you to a new file. Put all the required this in this format based on the required SRAM memory. (Word size, number of words and the technology to be used i.e. Sky130 or freepdk45 or any other needs to be mentioned. Add where you want to save all the output files (here a new folder named 'output') and also mention the files prefix needed (here 'sram_1kb'). Save and Exit file.*

```
abhishek@abhishek-VirtualBox:~/OpenRAM$ nano sram_1kb.py
```

abhishek@abhishek-VirtualBox: ~/OpenRAM

```
  GNU nano 6.2                                          sram_1kb.py *
word_size = 8                   # Number of bits per word
num_words = 1024                # Total number of words
tech_name = "freepdk45"         # Use FreePDK45 instead of sky130

output_path = "output"          # Directory to store all output
output_name = "sram_1kb"        # Prefix for output files

process_corners = ["TT"]        # Typical process corner
supply_voltages = [1.0]         # Nominal VDD for FreePDK45
temperatures = [25]             # Room temperature

nominal_corner_only = True
```

7. NOTE: There is one line of code you will need to remove from the compiler modules (this is necessary for the proper generation of your output files). To do this do to the OpenRAM repository, in there open /compiler, in there open /module. This can be done by typing the code : **cd compiler/modules/**

This will open all the modules available. You have to edit the file named **'__init__.py'**. *So, open the file using the command* **nano __init__.py**

```
abhishek@abhishek-VirtualBox:~/OpenRAM$ nano sram_1kb.py
abhishek@abhishek-VirtualBox:~/OpenRAM$ cd compiler/modules/
abhishek@abhishek-VirtualBox:~/OpenRAM/compiler/modules$ ls
and2_dec.py                    dummy_array.py                 pdriver.py                     rom_column_mux_array.py
and3_dec.py                    dummy_bitcell_1port.py         pgate.py                       rom_column_mux.py
and4_dec.py                    dummy_bitcell_2port.py         pinvbuf.py                     rom_control_logic.py
bank.py                        dummy_pbitcell.py              pinv_dec.py                    rom_decoder.py
bitcell_1port.py               global_bitcell_array.py        pinv.py                        rom_poly_tap.py
bitcell_2port.py               hierarchical_decoder.py        pnand2.py                      rom_precharge_array.py
bitcell_array.py               hierarchical_predecode2x4.py   pnand3.py                      rom_precharge_cell.py
bitcell_base_array.py          hierarchical_predecode3x8.py   pnand4.py                      rom_wordline_driver_array.py
bitcell_base.py                hierarchical_predecode4x16.py  pnor2.py                       row_cap_array.py
capped_replica_bitcell_array.py hierarchical_predecode.py     port_address.py                row_cap_bitcell_1port.py
col_cap_array.py               __init__.py                    port_data.py                   row_cap_bitcell_2port.py
col_cap_bitcell_1port.py       internal_base.py               precharge_array.py             sense_amp_array.py
col_cap_bitcell_2port.py       inv_dec.py                     precharge.py                   sense_amp.py
column_decoder.py              local_bitcell_array.py         ptristate_inv.py               sram_1bank.py
column_mux_array.py            multibank.py                   ptx.py                         sram_multibank.py
column_mux.py                  multi_delay_chain.py           pwrite_driver.py               sram_multibank_template.v
control_logic_base.py          nand2_dec.py                   replica_bitcell_1port.py       template.py
control_logic_delay.py         nand3_dec.py                   replica_bitcell_2port.py       tri_gate_array.py
control_logic.py               nand4_dec.py                   replica_bitcell_array.py       tri_gate.py
delay_chain.py                 orig_bitcell_array.py          replica_column.py              wordline_buffer_array.py
dff_array.py                   pand2.py                       replica_pbitcell.py            wordline_driver_array.py
dff_buf_array.py               pand3.py                       rom_address_control_array.py   wordline_driver.py
dff_buf.py                     pand4.py                       rom_address_control_buf.py     write_driver_array.py
dff_inv_array.py               pbitcell.py                    rom_bank.py                    write_driver.py
dff_inv.py                     pbuf_dec.py                    rom_base_array.py              write_mask_and_array.py
dff.py                         pbuf.py                        rom_base_cell.py
abhishek@abhishek-VirtualBox:~/OpenRAM/compiler/modules$ __init__.py
__init__.py: command not found
abhishek@abhishek-VirtualBox:~/OpenRAM/compiler/modules$ nano __init__.py
abhishek@abhishek-VirtualBox:~/OpenRAM/compiler/modules$ S
```

*In '__init__.py' comment out the code line ' from .rom_bank import *'. Save and exit this file. (Ctrl+O and Enter to Save, Ctrl +X to exit the file).*

```
from .replica_column import *
from .replica_pbitcell import *
from .row_cap_array import *
from .row_cap_bitcell_1port import *
from .row_cap_bitcell_2port import *
#from .rom_bank import *
from .sense_amp_array import *
from .sense_amp import *
from .tri_gate_array import *
from .tri_gate import *
from .wordline_buffer_array import *
from .wordline_driver_array import *
from .wordline_driver import *
from .write_driver_array import *
from .write_driver import *
from .write_mask_and_array import *
from .sram_1bank import *
from .internal_base import *
```

8. Set environment variables. Environment variables tell OpenRAM where to find its technology files (PDKs), configuration, and other dependencies. (IMPORTANT STEP)

Code:

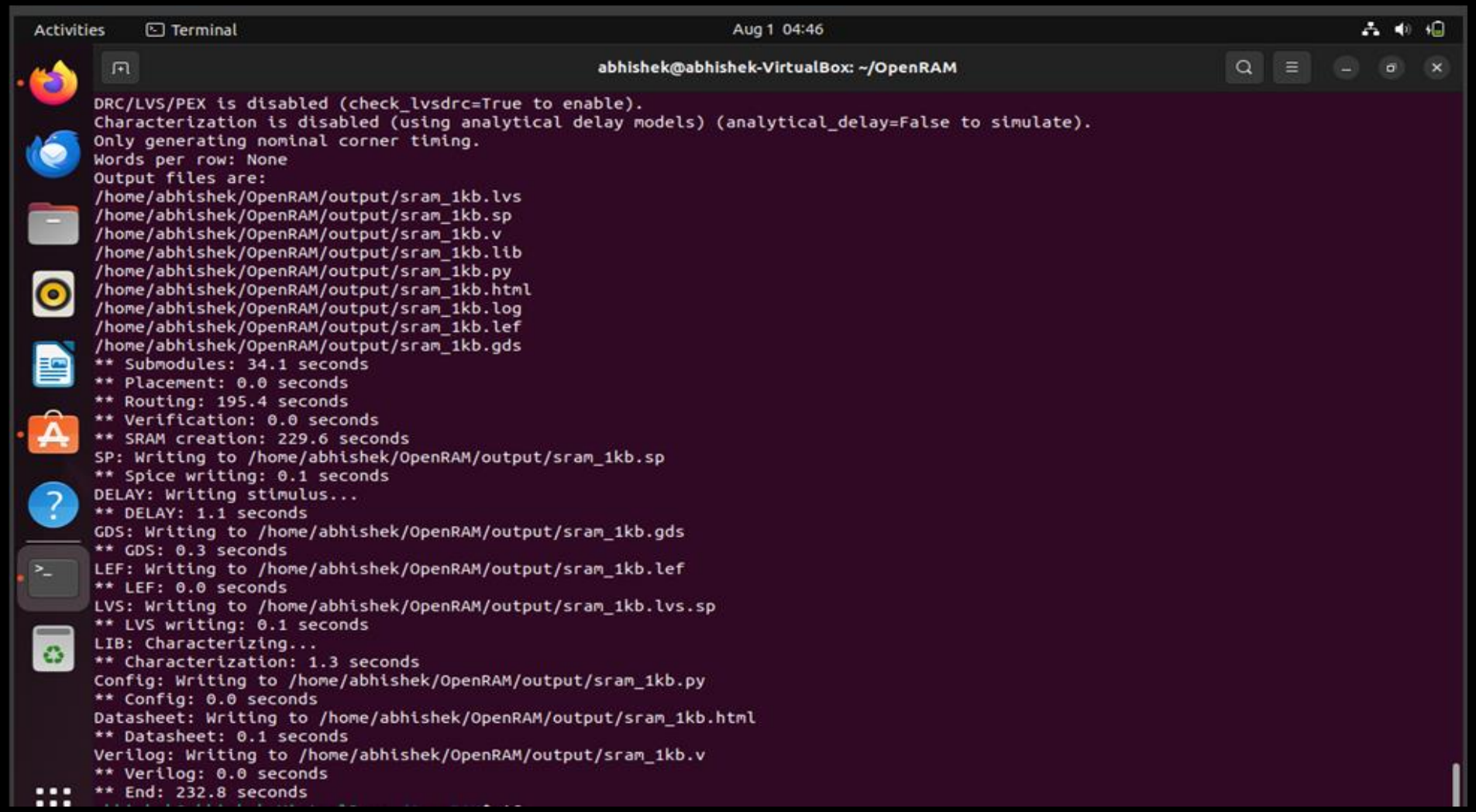
export OPENRAM_HOME="$HOME/OpenRAM/compiler"
export OPENRAM_TECH="$HOME/OpenRAM/technology"
export PYTHONPATH="$OPENRAM_HOME:$OPENRAM_TECH"

```
abhishek@abhishek-VirtualBox:~/Desktop$ cd ../
abhishek@abhishek-VirtualBox:~$ export OPENRAM_HOME="$HOME/OpenRAM/compiler"
export OPENRAM_TECH="$HOME/OpenRAM/technology"
export PYTHONPATH="$OPENRAM_HOME:$OPENRAM_TECH"
```

9. Open OpenRAM using: **cd openRAM/** *and then ask it to generate the output files using the command:* **python3 sram_compiler.py sram_1kb.py**

```
abhishek@abhishek-VirtualBox:~$ cd OpenRAM/
abhishek@abhishek-VirtualBox:~/OpenRAM$ python3 sram_compiler.py sram_1kb.py
WARNING: file globals.py: line 168: Failed to find coverage installation. This can be installed with pip3 install coverage
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 19: curl: command not found
/bin/bash: miniconda_installer_py38.sh: No such file or directory
rm: cannot remove 'miniconda_installer_py38.sh': No such file or directory
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 22: /home/abhishek/OpenRAM/miniconda/bin/activate: No such file or directo
ry
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 25: conda: command not found
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 26: conda: command not found
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 29: conda: command not found
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 34: conda: command not found
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 34: conda: command not found
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 34: conda: command not found
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 34: conda: command not found
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 34: conda: command not found
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 34: conda: command not found
Defaulting to user installation because normal site-packages is not writeable
Collecting scikit-learn>=0.22.2
```

It will take about 3-5 minutes to generate the output files (this is for 1kb memory, more the memory more time it will take for output generation)



It will take about 3-5 minutes to generate the output files (this is for 1kb memory, more the memory more time it will take for output generation)

.....continued

```
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed anyio-4.9.0 argparse-1.4.0 certifi-2025.7.14 ciel-2.0.3 click-8.2.1 coverage-7.10.1 exceptiongroup-1.3.0 h11-
0.16.0 httpcore-1.0.9 httpx-0.28.1 idna-3.10 iso8601-2.1.0 joblib-1.5.1 linecache2-1.0.0 markdown-it-py-3.0.0 mdurl-0.1.2 numpy-2.2.
6 pcpp-1.30 pygments-2.19.2 python-subunit-1.4.4 pyyaml-6.0.2 rich-14.1.0 scikit-learn-1.7.1 scipy-1.15.3 six-1.17.0 sniffio-1.3.1 t
esttools-2.7.2 threadpoolctl-3.6.0 traceback2-1.4.0 typing_extensions-4.14.1 unittest2-1.1.0 zstandard-0.23.0
/home/abhishek/OpenRAM/compiler/../install_conda.sh: line 41: conda: command not found
|==============================================================================|
|=========                      OpenRAM v1.2.49                       =========|
|=========                                                            =========|
|=========               VLSI Design and Automation Lab               =========|
|=========        Computer Science and Engineering Department         =========|
|=========            University of California Santa Cruz             =========|
|=========                                                            =========|
|=========          Usage help: openram-user-group@ucsc.edu           =========|
|=========        Development help: openram-dev-group@ucsc.edu        =========|
|=========                See LICENSE for license info                =========|
|==============================================================================|
** Start: 08/01/2025 04:39:50
Technology: freepdk45
Total size: 8192 bits
Word size: 8
Words: 1024
Banks: 1
RW ports: 1
R-only ports: 0
W-only ports: 0
DRC/LVS/PEX is only run on the top-level design to save run-time (inline_lvsdrc=True to do inline checking).
```

10. The output folder is generated. Now you can use all these different types of files for various purposes using various other tools. The different output files generated are shown below.

```
abhishek@abhishek-VirtualBox:~/OpenRAM$ ls
CNAME          docker          LICENSE                    openram.mk     README.md        sram_1kb.py      VERSION
common.py      docs            macros                     output         requirements.txt sram_char.py
compiler       images          Makefile                   PORTING.md     rom_compiler.py  sram_compiler.py
_config.yml    __init__.py     MANIFEST.in                __pycache__    setpaths.sh      sram_func.py
CONTRIBUTING.md install_conda.sh OpenRAM_ICCAD_2016_paper.pdf pyproject.toml setup.py      technology
abhishek@abhishek-VirtualBox:~/OpenRAM$ cd output/
abhishek@abhishek-VirtualBox:~/OpenRAM/output$ ls
datasheet.info functional_meas.sp sram_1kb.html sram_1kb.lvs.sp sram_1kb_TT_1p0V_25C.lib trimmed.sp
delay_meas.sp  functional_stim.sp sram_1kb.lef  sram_1kb.py     sram_1kb.v
delay_stim.sp  sram_1kb.gds      sram_1kb.log  sram_1kb.sp     sram.sp
```

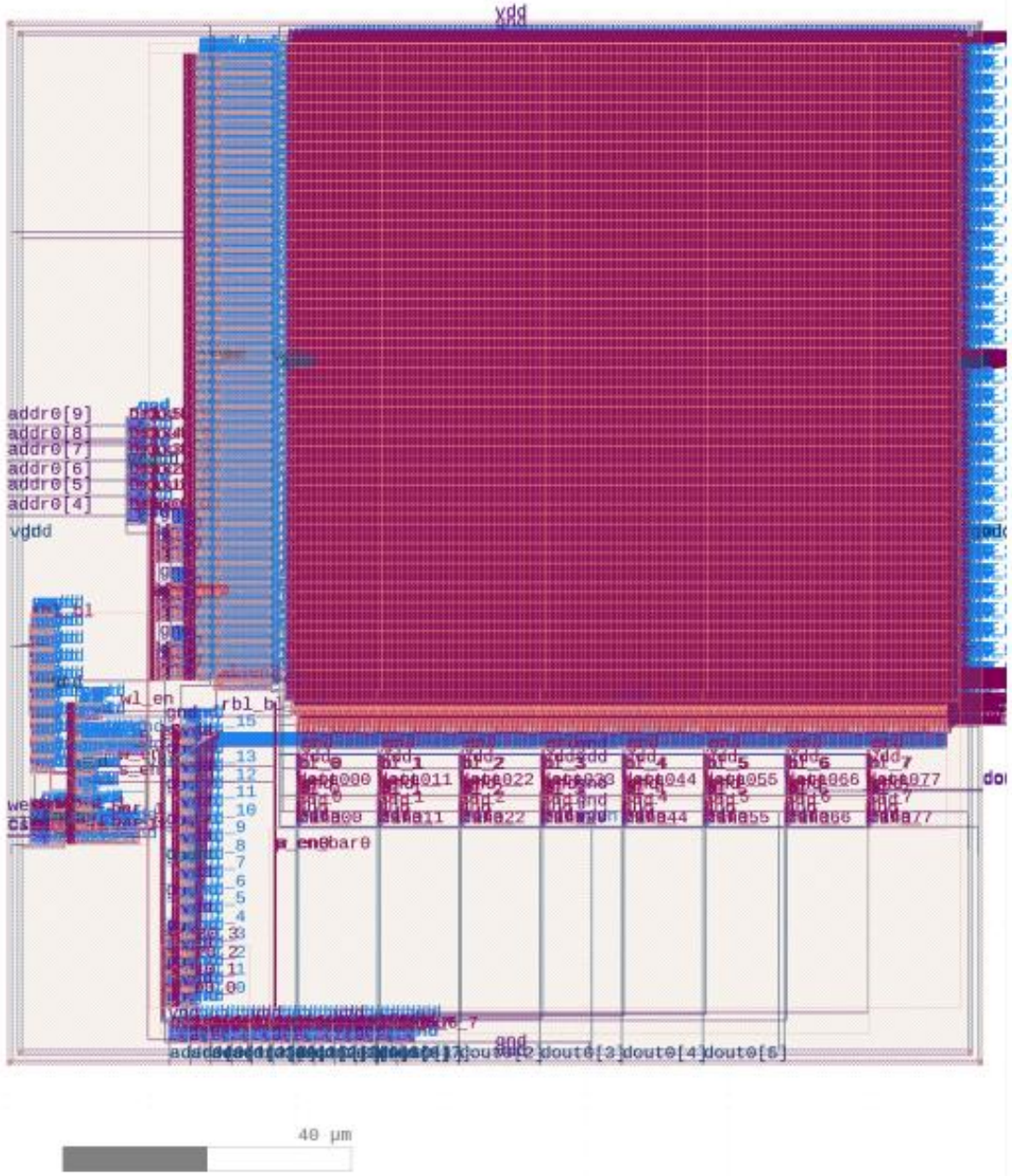
11. Install Klayout software. KLayout is powerful open-source tool used to view and verify the physical layout (GDSII files) that OpenRAM generates.

Code:

sudo apt install klayout

```
abhishek@abhishek-VirtualBox:~/Desktop$ cd ../
abhishek@abhishek-VirtualBox:~$ cd OpenRAM/
abhishek@abhishek-VirtualBox:~/OpenRAM$ cd output/
abhishek@abhishek-VirtualBox:~/OpenRAM/output$ klayout sram_1kb.gds
ERROR: :/built-in-macros/lvs_interpreters.lym:62: Operation cancelled in Application::instance
  :/built-in-macros/lvs_interpreters.lym:62:in `instance'
  :/built-in-macros/lvs_interpreters.lym:62:in `initialize'
  :/built-in-macros/lvs_interpreters.lym:130:in `new'
  :/built-in-macros/lvs_interpreters.lym:130:in `<module:LVS>'
  :/built-in-macros/lvs_interpreters.lym:2:in `<main>' (class RuntimeError)
ERROR: :/built-in-macros/drc_install.lym:6: Operation cancelled in Application::instance
  :/built-in-macros/drc_install.lym:6:in `instance'
  :/built-in-macros/drc_install.lym:6:in `<module:DRC>'
  :/built-in-macros/drc_install.lym:2:in `<main>' (class RuntimeError)
ERROR: :/built-in-macros/lvs_install.lym:6: Operation cancelled in Application::instance
  :/built-in-macros/lvs_install.lym:6:in `instance'
  :/built-in-macros/lvs_install.lym:6:in `<module:LVS>'
  :/built-in-macros/lvs_install.lym:2:in `<main>' (class RuntimeError)
```

Klayout is used to visualize the GDS file. Once you type: **klayout sram_1kb.gds** *in the terminal it opens your GDS file in a new Klayout window.*



This is a klayout GDS file for reference. Many such files are generated and can be worked with.